

Project Two: Summary and Reflection

CS 320 Software Test, Automation QA

Southern New Hampshire University

Cammy Holmes

June 20 2026

Summary

For this project, I developed three main features: the contact service, task service, and appointment service. For each of these, I used a unit testing approach where I tested individual pieces of functionality using JUnit. My goal was to make sure that each class and method worked correctly on its own before thinking about the system as a whole. For example, in the contact service, I tested that each field followed the required rules, such as ensuring the contact ID was not null and did not exceed 10 characters. In the task service, I tested adding, updating, and deleting tasks. In the appointment service, I tested that appointment dates could not be set in the past by using a future date and comparing it to the current time.

My testing approach was closely aligned with the software requirements because I created tests that directly matched each requirement. For example, when the requirements stated that a task ID must be unique, I tested adding duplicate entries and confirmed that an exception was thrown. I used `assertThrows(IllegalArgumentException.class, () -> { service.addTask(task); })` to verify this behavior. I also tested valid cases to make sure the system worked as expected under normal conditions. This approach ensured that both success and failure scenarios were covered.

I believe the overall quality of my JUnit tests was strong because I covered both valid inputs and edge cases. While I did not calculate an exact coverage percentage, I made sure every requirement had at least one test associated with it. For example, I used `assertEquals` to confirm that updates worked correctly and `assertNotNull` to verify that objects were successfully added. This level of testing typically results in high coverage for small applications like this, likely meeting the 80% requirement.

Writing JUnit tests was a learning experience. At first, it felt repetitive, but I started to see how each test connected directly to a requirement. Over time, it became more natural to think about how code might fail instead of just assuming it would work. This shift in thinking helped me write better tests and better code overall.

To ensure that my code was technically sound, I used validation checks in my constructors and setter methods and then confirmed through testing. For example, I used `assertThrows(IllegalArgumentException.class, () -> { new Task(null, "Name", "Desc"); });` to verify that the invalid inputs were rejected. I also used `assertEquals("New", service.getTask("1").getName());` to confirm that the updates were applied correctly. These tests helped ensure that my code behaved consistently and met requirements.

I also focused on keeping my code efficient by using simple data structures and clear logic. I used HashMaps to store contacts, tasks, and appointments, which allowed for fast lookups and updates. In my tests, I focused on one behavior at a time, which made it easier to identify issues and kept the code clean and readable.

Reflection

Testing Techniques

The main testing technique I used in this project was unit testing. Unit testing focuses on testing small, individual parts of a program to make sure they function correctly. This includes testing methods, constructors, and validation logic. I used JUnit to implement these tests, which allowed me to automate the testing process and verify expected outcomes using assertions.

There are several other testing techniques that I did not use in this project. Integration testing focuses on how different parts of a system works together, while system testing evaluates the entire application as a whole. Performance testing checks how a system behaves under heavy use, and security testing looks for vulnerabilities. I also did not use test-driven development (TDD), where tests are written before the code.

Each of these techniques has practical uses depending on the situation. Unit testing is ideal for catching issues early and ensuring that individual components work correctly. Integration testing becomes important in larger systems where multiple components interact. System testing is useful before deployment to confirm that the entire system functions as expected. Performance and security testing are important for applications that handle large amounts of data or sensitive information.

Mindset

While working on this project, I started to adopt more of a testers mindset instead of just a developers mindset. I focused on being cautious and thinking about how the code could fail, not just how it should work. For example, instead of only testing valid inputs, I also tested null values, incorrect lengths, and duplicate entries. This helped me catch potential issues early and made the code more reliable.

It was important to understand the relationship between different parts of the code. For example, the service classes depended on the object classes, so if validation was missing in one place, it could affect the entire system. Recognizing these connections helped me approach testing more carefully and thoughtfully.

I also tried to limit bias when testing my code. As the developer, it is easy to assume that my code works as intended, but this is not always the case. By writing tests that expected failures, such as using `assertThrows`, I was able to challenge my own assumptions. This helped reduce bias and made my testing more objective.

Being disciplined about quality is important in software development. Cutting corners on testing can lead to bigger problems later, especially as projects grow in size and complexity. I learned that taking the time to write proper tests helps prevent bugs and reduces the need for major fixes later. This also helps avoid technical debt, which can slow down development and makes systems harder to maintain. Moving forward, I plan to continue using structured testing approaches and focusing on writing clean, reliable code from the start.